Capitolo 1

Heaps

1.1 Problemi su Max-Heaps

Esercizio 1.1.1 #MaxHeap

Sia H un max-heap di dimensione n e sia $H[k]$ la chiave memorizzata in posizione k con $1 \leq k \leq n$ da decrementare di una unità. Si proponga un algoritmo che decrementi $H[k]$ e restituisca H modificato (Si noti, dopo la modifica, lo heap deve preservare le proprie caratteristiche di max-heap). Dare lo pseudocodice dell'algoritmo. Studiare la complessità in tempo della soluzione proposta.

Elaborazione Soluzione: Poiché $H[k]$ decresce, la nuova chiave può solo scendere nel max-heap, inoltre è la sola chiave che può violare le proprietà dello heap, quindi applichiamo la funzione MAXHEAPFY affinché ricollochi correttamente l'elemento in $H[k]$, così che lo scambi con uno più in basso.

DecrementaPosizione(**Array** $H[n]$, **int** k)
// H è l'heap e k la posizione in cui vi è l'elemento da decrementare
// n è la dimensione del heap
$H[k]--$
MAXHEAPFY(H, k) // Funzione che mantiene la proprietà dello heap in posizione k

Per completezza riportiamo la funzione MAXHEAPFY:

maxHeapfy(**Array** $H[n]$, **int** k)
int $max = 0$
$left = 2k$ // Il figlio sinistro è in questa posizione
$right = 2k + 1$ // Il figlio destro è in questa posizione
if $left(k) < n$ **then**
  if $H[k] < H[left]$ **then**
    $max = left$
  else
    $max = k$
if $right < n$ **then**
  if $H[max] < H[right]$ **then**
    $max = right$
// max contiene l'indice del nodo in $k, 2k, 2k + 1$ con chiave maggiore
if $max \neq k$ **then**
  SCAMBIA$(H[max], H[k])$
  maxHeapfy(H, max)

Complessità: MAXHEAPFY ha complessità $O(\log n)$, ma, poiché la applichiamo solo dalla posizione k in poi, sottraiamogli l'altezza del nodo k, ossia $O(\log k)$, ottenendo il costo $O(\log n - \log k) = O(\log \frac{n}{k})$.

Esercizio 1.1.2 #MaxHeap #Ottimalità

Dato un max-heap H di dimensione n, progettare un algoritmo che cerca il minimo in H.
Progettare e dare lo pseudocodice dell'algoritmo proposto.
Studiare la complessità dell'algoritmo proposto.
Discutere l'ottimalità della soluzione proposta.

Elaborazione Soluzione: Sappiamo che in un max-heap il valore minimo è in una delle foglie. Sappiamo che un heap per via della rigidità della sua struttura può essere visto come un array; supponiamo che sia quindi memorizzato in tale struttura dati. Sappiamo anche che in un albero al massimo metà dei nodi

possono essere foglie; essendo un max-heap allora il minimo sarà, nell'array che lo memorizza, tra la posizione $\lfloor \frac{n}{2} \rfloor + 1$ e la posizione n.
Quindi basta fare una ricerca del minimo in quella porzione dell'array.

Esempio: Heap di 7 elementi con le foglie in posizione 4, 5, 6, 7; il minimo può stare in una qualsiasi foglia.

10, 4, 6, 1, 3, 2, 5
10, 4, 6, 3, 1, 2, 5
10, 4, 6, 3, 2, 1, 5
10, 4, 6, 3, 2, 5, 1

Quindi l'algoritmo pensato è ottimo.

Pseudocodice:

Find-Min(**Array** $H[n]$)
int $start = \lfloor \frac{n}{2} \rfloor + 1$
int $min = H[start]$
for $i = start + 1$, $i \leq n$, $i++$ **do**
  if $H[i] < min$ **then**
    $min = H[i]$
return: min

Complessità ed Ottimalità: Questa soluzione ha costo $\lfloor \frac{n}{2} \rfloor + 1 \in O(n)$. Per stimare la complessità intrinseca del problema: l'albero delle decisioni suggerisce $\Omega(\log n)$. Tuttavia sappiamo che il problema si riduce a cercare il minimo di un array (senza alcun ordinamento) con $\frac{n}{2}$ elementi. Quindi abbiamo anche un lower bound più stretto pari a $\Omega(n)$. L'algoritmo proposto è perciò ottimo.

Esercizio 1.1.3 #MaxHeap

Dato un max-heap H i cui nodi contengono interi, ed un intero $1 \leq k \leq |H|$ proporre un algoritmo per raddopiare il valore di $H[k]$ e restituire H modificato.
Analizzare la complessità in tempo della soluzione proposta.

N.B. H deve soddisfare le proprietà del max-heap dopo le modifiche.

Elaborazione Soluzione: Essendo in un max-heap, il nuovo valore di $H[k]$ può solo risalire, fino al più alla radice.

Pseudocodice:

maxHeapfy(**Array** $H[n]$, **int** k)
$H[k] = 2 \cdot H[k]$
int $padre = \lfloor \frac{k}{2} \rfloor$
int $figlio = k$
while $padre \geq 1$ **and** $H[padre] < H[figlio]$ **do**
  SCAMBIA($H[padre], A[figlio]$
  $figlio = padre$
  $padre = \lfloor \frac{padre}{2} \rfloor$

Complessità: La complessità è $\Theta(\log(k))$ ossia l'altezza nello heap del nodo k.

Esercizio 1.1.4 #MaxHeap #Correttezza

Dato un vettore A di n elementi, ordinato in senso crescente, si trasformi A in modo che dopo la trasformazione esso rappresenti un max-heap. Si scriva lo pseudo-codice per tale trasformazione.
Si provi la correttezza del codice.
Si discuta la complessità in tempo e in spazio dell'algoritmo proposto.

Elaborazione Soluzione: Un array ordinato in senso decrescente è un max-heap (non vale il contrario, i max-heap non sono sempre array ordinati in senso decrescente), quindi basta capovolgere l'array.

Correttezza: Poiché ogni array ha la forma di uno heap e un array ordinato in senso decrescente soddisfa la max-heap property, la soluzione è corretta.

Pseudocodice:

Array-Rotation(**Array** $A[n]$)
int $t = 0$ // Variabile temporanea
for $i = 1$, $i \leq \lceil \frac{n}{2} \rceil$, $i++$ **do**
    $t = A[i]$
    $A[i] = A[n + 1 - i]$
    $A[n + 1 - i] = t$

Complessità: L'algoritmo realizzato ha costa spaziale $O(1)$, perché non viene allocata nuova memoria; si esegue il capovolgimento direttamente sull'array originale. Il costo temporale è di $O(n)$, perché leggiamo l'array fino alla metà ($\lceil \frac{n}{2} \rceil \in O(n)$); questo costo è anche ottimo, perché è quello per la creazione di un heap.

1.2 Problemi su Min-Heaps

Esercizio 1.2.1 #MinHeap

Dati un min-heap A i cui elementi sono interi, distinti e non negativi, e dato un intero k, progettare un algoritmo per restituire gli elementi di A minori o uguali a k.
Dare lo pseudocodice della soluzione e discutere la complessità della soluzione proposta.

Elaborazione Soluzione: In un min-heap abbiamo alla radice il valore minore ed alle foglie quelli maggiori. I min-heap sono alberi che per la loro rigidità è possibile memorizzarli in forma di array. Un min-heap non è un array ordinato in senso crescente (ma un array ordinato in senso crescente è un min-heap), ma, per la heap-property, se un elemento $A[x] > k$, tutti i suoi discendenti sono maggiori di k e non debbono essere ispezionati.

Quindi per risolvere il problema basta visitare lo heap come un albero binario e potare la visita ogni volta che si incontra una chiave maggiore di k.

Pseudocodice:

```
Find-Mins(Array A[n])
int i = 1
int sol[n] = ∅
int Q[n] = ∅
ENQUEUE(Q, A[1], 1)
while Q ≠ ∅ do
    (value, pos) = DEQUEUE(Q)
    if value ≤ k then
        sol = sol ∪ value
        if 2pos ≤ n then
            ENQUEUE(Q, A[2pos], 2pos)
        if 2pos + 1 ≤ n then
            ENQUEUE(Q, A[2pos + 1], 2pos + 1)
return: sol
```

Complessità: L'algoritmo scorre oltre ai nodi $A[i] \leq k$, che costituiscono un ovvio lower bound al problema, al più 2 nodi (i figli) per ogni nodo nell'insieme Sol. Pertanto la complessità è $O(|Sol| + 2|Sol|)$. L'algoritmo è ottimo.

Esercizio 1.2.2 #MinHeap

Dato un min-heap H di dimensione n ed una posizione k, progettare un algoritmo che sostituisce ad $H[k]$ la chiave $H[k] - c$ e riordina, se necessario, lo heap.
Discutere la complessità in tempo dell'algoritmo proposto.

Elaborazione Soluzione: Se $c > 0$, si tratta di una DECREASE-KEY. La chiave può solo risalire verso la radice. Se $c < o$, si tratta di una INCREASE-KEY e la chiave scende. Lo heap è riordinato invocando HEAPIFY sulla posizione k.

Pseudocodice:

```
Chage-Key(Array H[n], int k, int c)
if c > 0 then
    H[k] = H[k] − c
    int i = k
    while A[⌊i/2⌋] > A[i] and i > 1 do
        SCAMBIA(A[⌊i/2⌋], A[i])
        i = ⌊i/2⌋
else
    HEAPIFY (H[n], k)
```

Complessità: Quando $c > 0$, la nuova chiave è più piccola della precedente, quindi può risalire al massimo da k fino alla radice (indice 1); la complessità è quindi $O(logk)$. Quando $c < 0$, la chiave scende e il costo è $O(\log \frac{n}{k})$.

Capitolo 2

Algoritmi su Alberi

2.1 Alberi Binari di Ricerca

Esercizio 2.1.1 #AlberoBinarioRicerca

Dato un albero binario di ricerca B i cui nodi contengono degli interi, restituire il successore dell'elemento minimo memorizzato in B. Descrivere un algoritmo che risolva il problema, darne lo pseudocodice e studiare la complessità dell'algoritmo proposto.

Elaborazione Soluzione: La peculiarità di questo esercizio è che x è il minimo in T. Per trovare il minimo in un albero di ricerca T, si scorre il cammino più a sinistra dell'albero T. Una volta trovato il minimo nel nodo x, il suo successore è il minimo del sotto-albero di destra se $x.right \neq$ NULL, oppure il padre ($x.p$). Nella procedura generale, quando x è un nodo generico e non ha figlio dx, si segue a ritroso il cammino dal nodo verso la radice fino a scoprire che il nodo su cui ci troviamo è figlio sinistro di suo padre. In questo caso, poiché x è il minimo in T, la condizione è subito verificata dal padre di x.

Pseudocodice:

```
Successore_Minimo(Nodo x)
while x.left ≠ NULL do
    x = x.left // Cerchiamo il minimo
// Vediamo se il successore è nel sotto-albero dx o il padre
if x.right ≠ NULL then
    x = x.right
    while x.left ≠ NULL do
        x = x.left
    return: x
else
    return: x.p

main()
// Controlliamo se l'albero non è vuoto
if T ≠ NULL then
    return: SUCCESSORE_MINIMO(T)
```

Complessità: $O(h)$

Esercizio 2.1.2 #AlberoBinarioRicerca

Dato un albero binario di ricerca T, ordinare in senso decrescente le chiavi in T.

- Dare lo pseudocodice dell'algoritmo
- Studiare la complessità dell'algoritmo proposto

Elaborazione Soluzione: Per risolvere l'esercizio osserviamo innanzitutto che la nostra struttura dati T è un albero binario di ricerca il quale ci garantisce le seguenti 3 proprietà:

1. Il sotto-albero sinistro di un nodo x contiene soltanto i nodi con chiavi minori della chiave del nodo x,
2. il sotto-albero destro di un nodo x contiene soltanto i nodi con chiavi maggiori o uguali alla chiave del nodo x, e
3. ogni sotto-albero è un albero binario di ricerca.

Sappiamo che una visita in-order ci restituirebbe tutti i nodi ordinati dal più piccolo al più grande del nostro albero binario di ricerca T. Invertendo la visita del sotto-albero sinistro e destro, restituiamo le chiavi in ordine decrescente.

Quindi, visitiamo il sotto-albero destro del nostro albero, dopodiché stampiamo la radice, ed infine visitiamo il sotto-albero sinistro.

Pseudocodice:

ReverseinorderTraversal(**Node** x, **Array** Arr)
if $root ==$ NULL **then**
  return: NULL
ReverseinorderTraversal($x.right$, Arr)
Arr.ADD(x)
ReverseinorderTraversal($x.left$, Arr)

Funzionamento: Lo pseudocodice descritto non fa altro che visitare ricorsivamente, in modo opposto alla visita in-order, il nostro albero binario di ricerca T. In particolare, i nodi vengono aggiunti all'interno di un array (Arr) nell'ordine desiderato sfruttando la nostra funzione REVERSEINORDERTRAVERSAL, e al termine dell'esecuzione l'array Arr conterrà tutti i nodi di T in ordine decrescente.

Complessità: Il costo della nostra funzione REVERSEINORDERTRAVERSAL sarà lo stesso di una visita in-order, e quindi asintoticamente $O(n)$, con n numero dei nodi del nostro albero T.
La soluzione è inoltre ottima in tempo perché tutte le chiavi devono essere visitate e quindi per la tecnica della dimensione dell'input, l'algoritmo è ottimo.

Osservazione: Si noti che un altra soluzione ottima poteva eseguire la INORDERTRAVERSAL e poi capovolgere sul posto (senza memoria aggiuntiva) l'array. Per capovolgere l'array sul posto, si inizializzano due cursori i e j, rispettivamente, al primo e ultimo elemento dell'array. Procedere scambiando il primo e l'ultimo elemento puntati, incrementare i, decrementare j fino a che $i \geq j$.

Esercizio 2.1.3 #AlberoBinarioRicerca

Dato un albero binario di ricerca T, i cui elementi sono interi, e due chiavi U ed L, stampare tutte le chiavi k tali che $L \leq k \leq U$.

- Dare lo pseudocodice dell'algoritmo
- Studiare la complessità dell'algoritmo proposto

Elaborazione Soluzione: Si fa una visita in-order, iniziando a stampare i nodi da L in poi fino ad U, per interrompere la visita dopo la stampa di U.

Pseudocodice:

Modified-in-order-visit(**Puntatore a Lista** x, **chiave** L, **chiave** U)
if $x \neq$ NULL **then**
  if $x.key < L$ **then**
    // Chiave $\leq$ di L, si va solo a destra alla ricerca di elementi maggiori
    Modified-in-order-visit($x.right, L, U$)
  else
    if $x.key > U$ **then**
      // Chiave $\geq$ di U, si va solo a sinistra alla ricerca di elementi minori
      Modified-in-order-visit($x.left, L, U$)
    else
      Modified-in-order-visit($x.left, L, U$)
      PRINT($x.key$)
      Modified-in-order-visit($x.right, L, U$)

Lancio dell'algoritmo: MODIFIED-IN-ORDER-VISIT(T, L, U)

Complessità: Qui l'algoritmo ignora i rami che non gli interessano, andando a cercare solo nei sotto-alberi dove può trovarsi la soluzione.

La complessità è $O(k+h)$ dove k è il numero delle chiavi memorizzate nell'albero comprese fra L e U; ed h è l'altezza dell'albero. Infatti, l'algoritmo impiega $O(h)$ per trovare L e U, per poi visitare tutte le chiavi comprese fra L e U.

Esercizio 2.1.4 #AlberoBinarioRicerca

Dato un albero binario di ricerca B i cui nodi contengono degli interi, restituire il predecessore dell'elemento massimo (ossia il secondo massimo) memorizzato in B.
Descrivere un algoritmo che risolva il problema e darne lo pseudocodice.
Studiare la complessità dell'algoritmo proposto.

Elaborazione Soluzione: Per trovare il massimo in un albero di ricerca T, si scorre il cammino più a destra dell'albero T. Una volta trovato il massimo nel nodo x (non ha più figli destri maggiori di lui), il suo predecessore è il massimo del sotto-albero di sinistra di x se $x.left \neq$ NULL, oppure la chiave memorizzata nel padre $(x.p)$.

Pseudocodice: Assumiamo che ogni nodo contenga un attributo puntatore al nodo padre.

```
Predecessore_Massimo(Nodo x)
while x.right ≠ NULL do
    x = x.right // Cerchiamo il massimo
/* Il predecessore del massimo è il massimo del sotto-albero radicato
nel figlio sinistro del nodo massimo o il padre del massimo */
if x.left ≠ NULL then
    x = x.left
    while x.right ≠ NULL do
        x = x.right // Cerchiamo il massimo
    return: x
else
    return: x.p
```

Lancio dell'algoritmo:

```
Main()
// Controlliamo se l'albero non è vuoto
if T ≠ NULL then
    y = PREDECESSORE_MASSIMO(T)
    return: y.key
```

Complessità: $O(h)$ dove h è l'altezza di T.

Esercizio 2.1.5 #AlberoBinarioRicerca #Correttezza

Sia T un albero binario di ricerca, progettate un algoritmo per ordinare in senso crescente le chiavi memorizzate in T.
Discutete la correttezza della soluzione proposta.
Discutete la complessità in tempo della soluzione proposta.

Elaborazione Soluzione: Dalla definizione di albero binario di ricerca, per ordinare i valori nei nodi di un albero binario di ricerca è sufficiente effettuare una visita in-order dell'albero.

Pseudocodice n.1: Caso in cui ordiniamo in senso crescente.

```
In-Order(Nodo x)
if x ≠ NULL then
    In-Order(x.left)
    STAMPA(x)
    In-Order(x.right)
```

Pseudocodice n.2: Caso in cui ordiniamo in senso decrescente.

```
In-Order(Nodo x)
if x ≠ NULL then
    In-Order(x.right)
    STAMPA(x)
    In-Order(x.left)
```

Complessità:La complessità è $\Theta(n)$, poiché tutti ed n i nodi devono essere visitati.

Esercizio 2.1.6 #AlberoBinarioRicerca

Il fattore di bilanciamento $b(v)$ di un nodo interno v di un albero binario T è definito come la differenza fra le altezze dei sotto-alberi radicati nel figlio sinistro e destro di v.
Progettare in pseudocodice un algoritmo per calcolare $b(v)$, $\forall v \in T$.
Discutere la complessità in tempo dell'algoritmo proposto.

Elaborazione Soluzione: Supponendo che i nodi dell'albero possano memorizzare sia la loro altezza che il loro fattore di bilanciamento, realizziamo un algoritmo che calcola le altezze dei nodi, per poi memorizzarle nel nodo. Poi modifichiamo una visita in-order (andava bene una visita qualsiasi) affinché per ogni nodo se ne imposti il fattore di bilanciamento conoscendo le altezze dei suoi sotto-alberi.

Pseudocodice:

```
Altezza(Nodo x)
if x ≠ NULL then
    x.altezza = 1 + MAX(altezza(x.left), altezza(x.right))
else
    return: 0

In-order-fat-bil(Nodo x)
if x ≠ NULL then
    In-order-fat-bil(x.left)
    if x.left ≠ NULL then
        if x.right ≠ NULL then
            x.fatBil = |x.left.altezza − x.right.altezza|
        else
            x.fatBil = x.left.altezza
    else
        if x.right ≠ NULL then
            x.fatBil = x.right.altezza
        else
            x.fatBil = 0
    In-order-fat-bil(x.right)
```

Lancio dell'algoritmo:

```
Main()
// Controlliamo se l'albero non è vuoto
if T ≠ NULL then
    ALTEZZA(T)
    IN-ORDER-FAT-BIL(T)
```

Complessità: L'algoritmo impiega $\Theta(n)$ sia per visitare l'intero albero ed impostare le altezze che per visitare l'intero albero per poi impostare i fattori di bilanciamento; quindi ha costo $\Theta(n)$.

Esercizio 2.1.7 #AlberoBinarioRicerca

Dato un albero binario di ricerca, realizzato con puntatori, i cui nodi contengono interi, sostituire il valore memorizzato in ciascun nodo con il suo predecessore.
Dare lo pseudocodice dell'algoritmo e studiare la complessità in tempo dell'algoritmo proposto.

Elaborazione Soluzione: Ricordando che la in-order visita di un albero binario di ricerca restituisce le chiavi ordinate in senso crescente, il precedessore di ogni chiave è la chiave del nodo visitato precedentemente. Il predecessore del minimo è fissato a $-\infty$

Pseudocodice:

In-Order(**Nodo** x, **int** $pred$)
if $x \neq$ NULL **then**
    In-Order$(x.left, pred)$
    $temp = x.key$
    $x.key = pred$
    $pred = temp$
    In-Order$(x.right, pred)$

Lancio dell'algoritmo: IN-ORDER$(r, -\infty)$

Complessità: Il costo della visita in ampiezza è $O(V + E)$, con V i nodi ed E gli archi; essendo un albero $E = V - 1$, quindi il costo è $O(V)$.
L'algoritmo impiega $\Theta(|V|)$ per visitare l'intero albero ed è ottimo perché ogni nodo deve essere visitato.

Osservazioni: Si noti che la variabile *pred* è passata modificata ad ogni chiamata ricorsiva. Si tratta un passaggio per indirizzo, per cui ad ogni chiamata si legge lo stesso indirizzo che memorizza *pred*. Pertanto, il valore di *pred* è modificato di chiamata in chiamata.